KARACHI INSTITUTE OF ECONOMICS AND TECHNOLOGY

COCIS

Department of Cyber Security

SECURE STUDENT PORTAL

A Web-Based Secure Application with Authentication, RBAC, and Attack Prevention

---

Subject: Secure Software Development

Submitted To: Miss Nazia Abbrar

Submitted By: Ibraheem Ahmed | Roll No: 15762

Semester: 6th

Date: May 2026

TABLE OF CONTENTS

1. Introduction

1.1 Project Overview

The Secure Student Portal is a web-based application developed as part of the Secure Software Development course at KIET. The project demonstrates the practical implementation of security principles in a real-world application. The portal provides a secure platform where students can view their academic marks and administrators can manage student records.

The application is built using Python Flask framework with SQLite database and Bootstrap for the frontend. Security was considered at every stage of development following the Secure Software Development Life Cycle (SSDLC).

1.2 Objectives

- Implement a secure authentication and authorization system
- Apply Role-Based Access Control (RBAC) with Admin and Student roles
- Preventing common web attacks: SQL Injection, XSS, CSRF, and Brute Force
- Implement security logging to track all user activities
- Follow Secure SDLC practices throughout development
- Demonstrate penetration testing on the developed application

1.3 Technology Stack

Component	Technology	Purpose
Backend	Python Flask	Web framework for routing and logic
Database	SQLite + SQLAlchemy	Data storage with ORM (prevents raw SQL)
Frontend	HTML + Bootstrap 5	Responsive user interface
Authentication	Flask-Login	Session management
Password Hashing	bcrypt	Secure password storage
CSRF Protection	Flask-WTF	Cross-site request forgery prevention

2. Secure Software Development Life Cycle (SSDLC)

The project followed the Secure SDLC model where security was integrated into every phase of development rather than being added as an afterthought.

Phase	Activities Performed
1. Planning	Identified security requirements, defined user roles (Admin/Student), set security goals
2. Design	Designed database schema, RBAC model, threat modeling using STRIDE framework
3. Development	Implemented secure coding: input validation, prepared queries, password hashing
4. Testing	Performed penetration testing: SQL Injection, XSS, CSRF, Brute Force attacks
5. Deployment	Configured debug mode, secret keys, and secure session management

3. STRIDE Threat Modeling

STRIDE is a threat modeling framework used to identify and categorize security threats. The following table shows how each STRIDE threat was identified and mitigated in the Secure Student Portal.

STRIDE	Threat	Example in Our App	Mitigation
S - Spoofing	Pretending to be someone else	Fake login as admin or student	Authentication with bcrypt hashing
T - Tampering	Unauthorized data modification	Students change their own marks	RBAC - only admin can modify marks
R - Repudiation	Denying performed actions	Admin denies adding wrong marks	Logging - all actions in portal.log
I - Info Disclosure	Sensitive data exposure	Student views another student's marks	Access control - own data only
D - Denial of Service	Making system unavailable	Brute force login attempts	Account lockout after 5 failed attempts
E - Elevation of Privilege	Gaining higher access	Student accesses /admin route	RBAC with role check on every route

4. System Design

4.1 Database Design

Users Table

Field	Type	Description
id	Integer (PK)	Unique identifier for each user
username	String(150)	Unique username for login
password	String(200)	bcrypt hashed password - never plain text
role	String(20)	User role: 'admin' or 'student'
failed_attempts	Integer	Counter for failed login attempts
is_locked	Boolean	Account lock status after 5 failed attempts

Marks Table

Field	Type	Description
id	Integer (PK)	Unique identifier for each mark record
subject	String(100)	Subject name
marks	Integer	Mark value between 0 and 100
student_id	Integer (FK)	Foreign key referencing users.id

4.2 Role-Based Access Control (RBAC)

Feature	Student	Admin
Login / Logout	Yes	Yes
View Own Marks	Yes	No
View All Students	No	Yes
Add Marks	No	Yes
Delete Marks	No	Yes
Unlock Accounts	No	Yes
Access /admin route	No (Access Denied)	Yes
Access /dashboard route	Yes	No (Redirected)

5. Security Features with Code

5.1 Authentication

The application implements a secure login system. Passwords are hashed using bcrypt before storing. Flask-Login manages secure session tokens and the @login_required decorator protects all sensitive routes.

Code: User Model (models defined in app.py)

```
class User(UserMixin, db.Model):
    __tablename__ = 'users'
    id              = db.Column(db.Integer, primary_key=True)
    username        = db.Column(db.String(150), unique=True, nullable=False)
    password        = db.Column(db.String(200), nullable=False)
    role            = db.Column(db.String(20), nullable=False,
default='student')
    failed_attempts = db.Column(db.Integer, default=0)
    is_locked       = db.Column(db.Boolean, default=False)
    marks           = db.relationship('Mark', backref='student', lazy=True)
```

5.2 Password Hashing with bcrypt

bcrypt is a one-way hashing algorithm with salt to prevent rainbow table attacks. It is intentionally slow to prevent brute force attacks. Plain text passwords are never stored.

Code: Password hashing on registration

```
# Hash the password before saving to database
hashed_pw = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())

new_user = User(
    username=username,
    password=hashed_pw.decode('utf-8'),  # Store hash, NOT plain text
    role=role
)
db.session.add(new_user)
db.session.commit()
```

Code: Password verification on login

```
# Compare entered password with stored hash
if user and bcrypt.checkpw(password.encode('utf-8'),
                           user.password.encode('utf-8')):
    login_user(user)  # Login successful
else:
    flash('Invalid username or password.', 'danger')
```

5.3 SQL Injection Prevention

SQL Injection is prevented through SQLAlchemy ORM which uses prepared statements internally. No raw SQL queries are used anywhere. All database operations go through the ORM.

Code: Safe database query using SQLAlchemy ORM (no raw SQL)

```
# SAFE - SQLAlchemy uses prepared statements internally
user = User.query.filter_by(username=username).first()

# The above is equivalent to a prepared statement:
# SELECT * FROM users WHERE username = ? LIMIT 1
# The ? is safely parameterized - SQL injection is impossible

# NEVER do this (raw SQL - vulnerable to injection):
```

```
# db.execute(f"SELECT * FROM users WHERE username = '{username}'")
```

5.4 XSS Prevention

Cross-Site Scripting is prevented through Jinja2 templating which automatically escapes all output. Any HTML or script tags are rendered as plain text and never executed.

Code: Jinja2 auto-escaping in templates (login.html)

```
{# Jinja2 auto-escapes all variables by default #}
{# If user enters <script>alert(1)</script> as username #}
{# It renders as plain text, NOT as executable code #}

<p>Welcome, {{ current_user.username }}</p>

{# Output: Welcome, <script>alert(1)</script>  (as text) #}
{# NOT:    Welcome, [popup alert]              (as code) #}
```

5.5 Brute Force Protection

The application tracks failed login attempts per user and locks the account after 5 consecutive failures. Admin can unlock accounts from the admin panel.

Code: Brute force protection in login route

```
# Check if account is already locked
if user and user.is_locked:
    flash('Account locked. Contact admin.', 'danger')
    return redirect(url_for('login'))

# Wrong password handling
if user:
    user.failed_attempts += 1
    if user.failed_attempts >= 5:
        user.is_locked = True
        db.session.commit()
        flash('Account locked after 5 failed attempts.', 'danger')
        return redirect(url_for('login'))
    db.session.commit()
    remaining = 5 - user.failed_attempts
    flash(f'Invalid password. {remaining} attempts remaining.', 'danger')

# Successful login - reset counter
user.failed_attempts = 0
user.is_locked = False
db.session.commit()
```

5.6 CSRF Protection

Cross-Site Request Forgery is prevented using Flask-WTF. Every form includes a hidden CSRF token. Requests without a valid token are automatically blocked.

Code: CSRF setup in app.py

```
from flask_wtf.csrf import CSRFProtect, CSRFError

csrf = CSRFProtect(app)  # Enables CSRF protection globally

# Handle CSRF errors
@app.errorhandler(CSRFError)
def handle_csrf_error(e):
    logger.warning(f'CSRF attack blocked: {e.description}')
    flash('Security error: Invalid CSRF token. Blocked.', 'danger')
    return redirect(url_for('login'))
```

Code: CSRF token in HTML form (login.html)

```
<form method="POST">
    <!-- Hidden CSRF token - must match server session token -->
    <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
    <input type="text" name="username" required>
    <input type="password" name="password" required>
    <button type="submit">Login</button>
</form>
```

5.7 Role-Based Access Control (RBAC)

Every route checks the user's role before allowing access. Students cannot access admin routes and admins are redirected away from student routes.

Code: RBAC role check on admin route

```
@app.route('/admin')
@login_required  # Must be logged in
def admin_dashboard():
    # Must be admin role
    if current_user.role != 'admin':
        flash('Access denied.', 'danger')
        return redirect(url_for('student_dashboard'))
    students = User.query.filter_by(role='student').all()
    return render_template('admin_dashboard.html', students=students)

@app.route('/dashboard')
@login_required  # Must be logged in
def student_dashboard():
    # Must be student role
    if current_user.role != 'student':
        flash('Access denied.', 'danger')
        return redirect(url_for('admin_dashboard'))
    marks = Mark.query.filter_by(student_id=current_user.id).all()
    return render_template('student_dashboard.html', marks=marks)
```

5.8 Security Logging

All security-relevant events are logged to portal.log with timestamps for audit trails and incident response.

Code: Logging setup in app.py

```
import logging

logging.basicConfig(
    filename='portal.log',
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)
logger = logging.getLogger('portal')
```

Code: Logging events throughout the application

```
# Successful login
logger.info(f'Successful login: {username} ({user.role})')

# Failed login attempt
logger.warning(f'Failed login attempt {user.failed_attempts}/5 for:
{username}')

# Account locked
logger.warning(f'Account LOCKED: {username}')

# New user registered
```

```
logger.info(f'New user registered: {username} as {role}')

# Admin added marks
logger.info(f'Admin {current_user.username} added mark: {subject} =
{marks_val}')

# CSRF attack blocked
logger.warning(f'CSRF attack blocked: {e.description}')
```

Event	Log Level	Example Log Entry
Successful login	INFO	2026-05-12 10:30:00 - INFO - Successful login: admin1 (admin)
Failed attempt	WARNING	2026-05-12 10:31:00 - WARNING - Failed login attempt 1/5 for: student1
Account locked	WARNING	2026-05-12 10:32:00 - WARNING - Account LOCKED: student1
New registration	INFO	2026-05-12 10:33:00 - INFO - New user registered: ali as student
Marks added	INFO	2026-05-12 10:34:00 - INFO - Admin added mark for student_id 2
CSRF blocked	WARNING	2026-05-12 10:35:00 - WARNING - CSRF attack blocked

6. Penetration Testing

Penetration testing was performed on the application to verify that all security measures work correctly.

6.1 SQL Injection Test

Attack Payload:

```
' OR 1=1 --
```

Entered in:

Username field on the login page

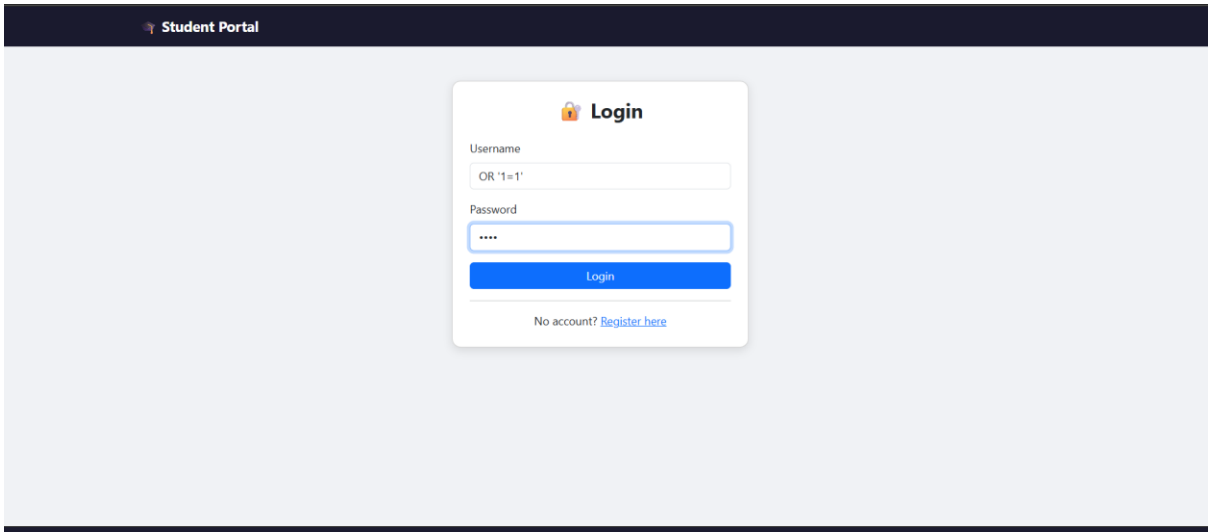
Expected Result:

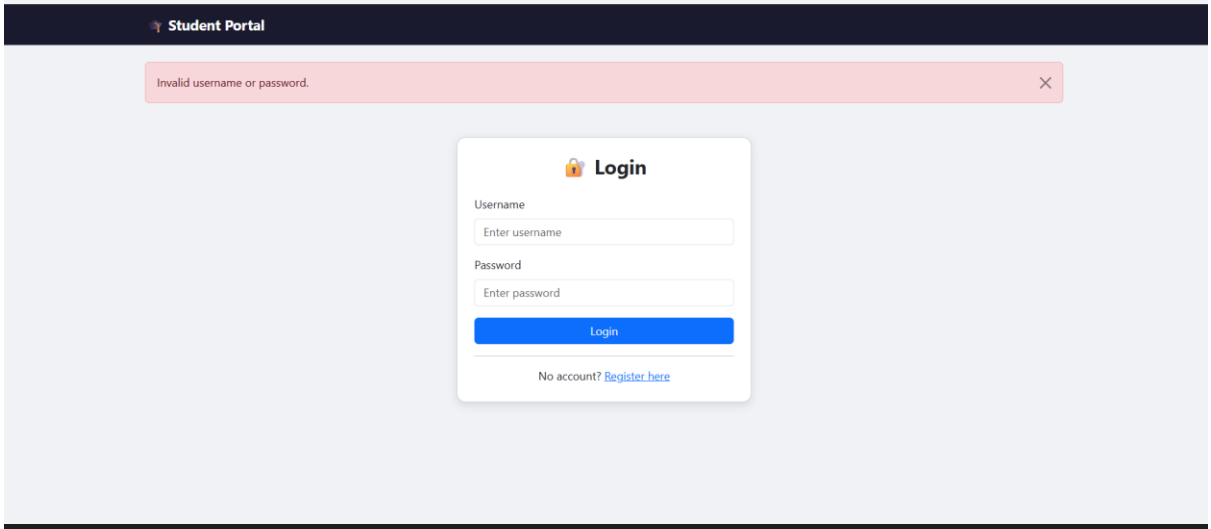
Login should be denied. The payload should not execute as SQL.

Actual Result:

Application returned 'Invalid username or password'. The attack was blocked because SQLAlchemy uses prepared statements. The payload was treated as a literal string, not SQL code.

STATUS: BLOCKED






```
3    :9 - INFO -  * Debugger PIN: 237-498-970
4    :9 - INFO - 127.0.0.1 - - [23/May/2026 15:08:34] "ESC[32mGET / HTTP/1.1ESC[0m" 302 -
5    :4 - INFO - 127.0.0.1 - - [23/May/2026 15:08:34] "GET /login HTTP/1.1" 200 -
6    :7 - WARNING - Failed login attempt for unknown user: OR '1=1'
7    :0 - INFO - 127.0.0.1 - - [23/May/2026 15:09:22] "ESC[32mPOST /login HTTP/1.1ESC[0m" 302 -
8    :1 - INFO - 127.0.0.1 - - [23/May/2026 15:09:22] "GET /login HTTP/1.1" 200 -
9
```

6.2 XSS Test

Attack Payload:

```
<script>alert(1)</script>
```

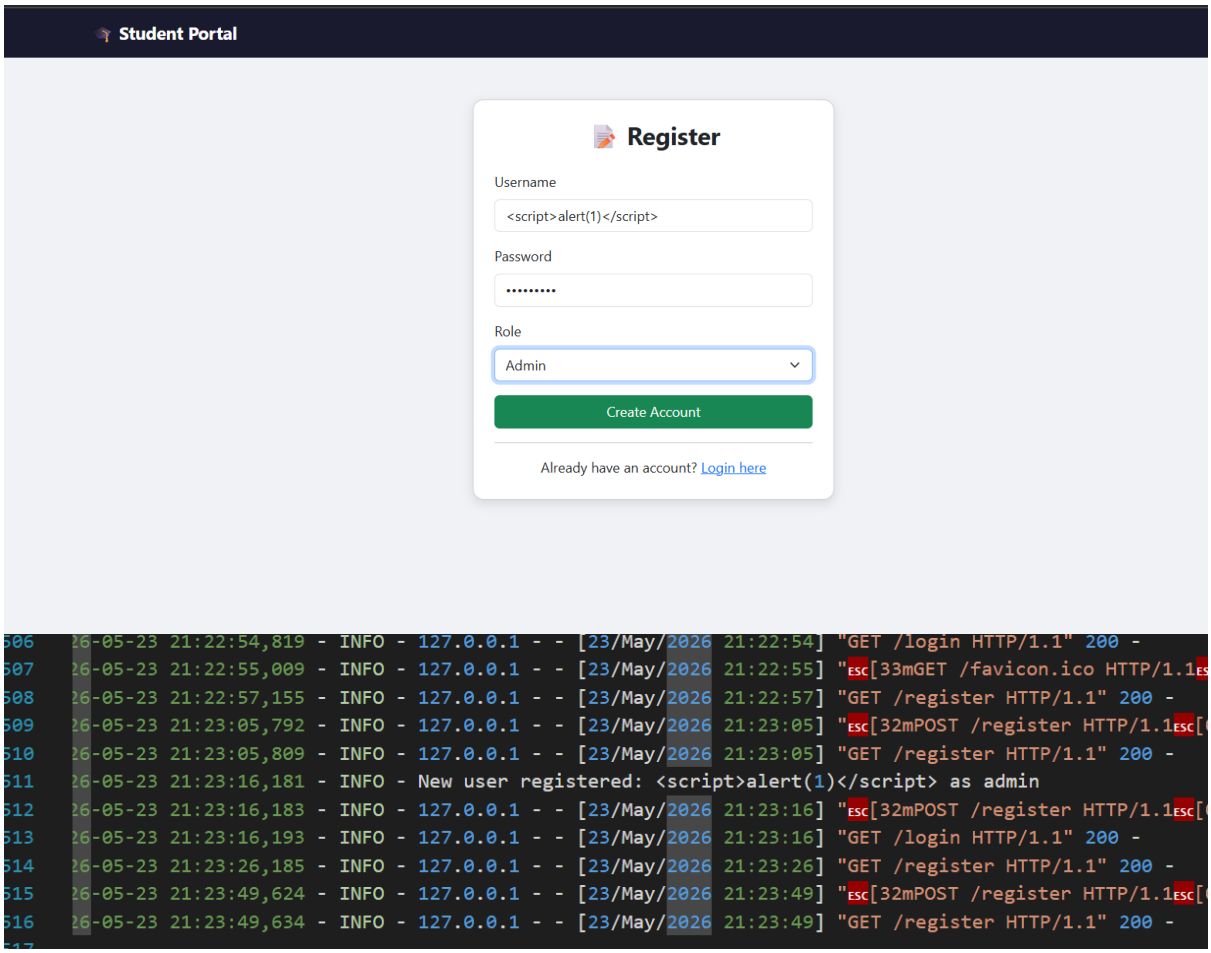
Entered in:

Username field on the register page

Actual Result:

Jinja2 auto-escaped the input. Script tags were rendered as plain text and no popup appeared.

STATUS: BLOCKED




```
506  26-05-23 21:22:54,819 - INFO - 127.0.0.1 - - [23/May/2026 21:22:54] "GET /login HTTP/1.1" 200 -
507  26-05-23 21:22:55,009 - INFO - 127.0.0.1 - - [23/May/2026 21:22:55] "←[33mGET /favicon.ico HTTP/1.1
508  26-05-23 21:22:57,155 - INFO - 127.0.0.1 - - [23/May/2026 21:22:57] "GET /register HTTP/1.1" 200 -
509  26-05-23 21:23:05,792 - INFO - 127.0.0.1 - - [23/May/2026 21:23:05] "←[32mPOST /register HTTP/1.1
510  26-05-23 21:23:05,809 - INFO - 127.0.0.1 - - [23/May/2026 21:23:05] "GET /register HTTP/1.1" 200 -
511  26-05-23 21:23:16,181 - INFO - New user registered: <script>alert(1)</script> as admin
512  26-05-23 21:23:16,183 - INFO - 127.0.0.1 - - [23/May/2026 21:23:16] "←[32mPOST /register HTTP/1.1
513  26-05-23 21:23:16,193 - INFO - 127.0.0.1 - - [23/May/2026 21:23:16] "GET /login HTTP/1.1" 200 -
514  26-05-23 21:23:26,185 - INFO - 127.0.0.1 - - [23/May/2026 21:23:26] "GET /register HTTP/1.1" 200 -
515  26-05-23 21:23:49,624 - INFO - 127.0.0.1 - - [23/May/2026 21:23:49] "←[32mPOST /register HTTP/1.1
516  26-05-23 21:23:49,634 - INFO - 127.0.0.1 - - [23/May/2026 21:23:49] "GET /register HTTP/1.1" 200 -
```

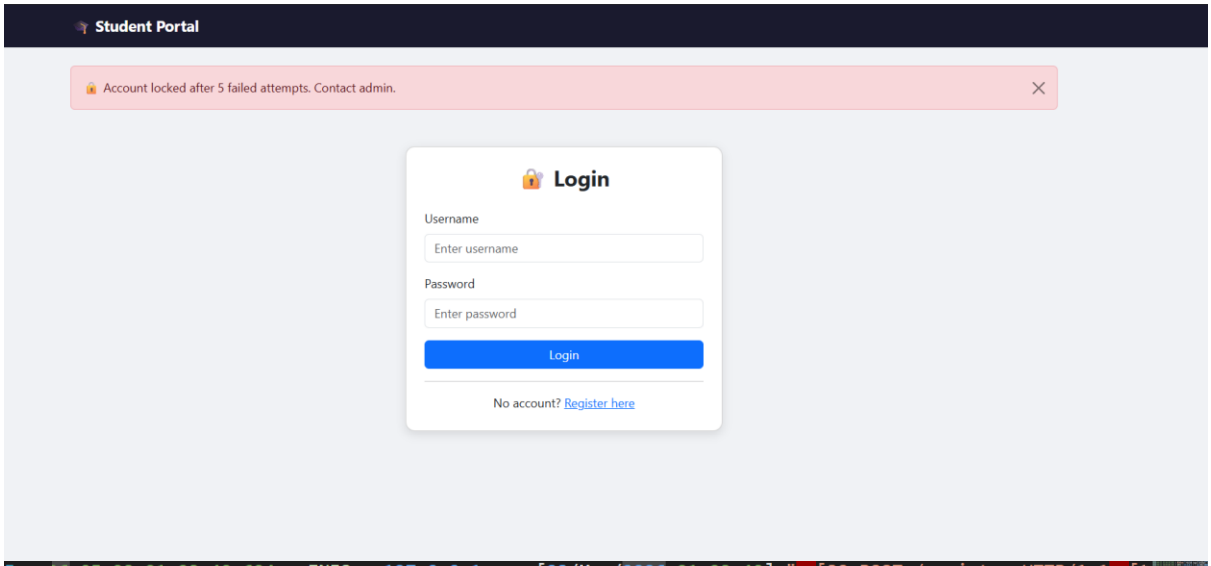
6.3 Brute Force Test

Method:

Entered wrong password repeatedly for a valid username.

Attempt	Password	System Response
1st	wrongpass	Invalid password. 4 attempts remaining before lockout.
2nd	wrongpass	Invalid password. 3 attempts remaining before lockout.
3rd	wrongpass	Invalid password. 2 attempts remaining before lockout.
4th	wrongpass	Invalid password. 1 attempts remaining before lockout.
5th	wrongpass	Account locked after 5 failed attempts. Contact admin.
6th	correctpass	Account is locked. Login denied even with correct password.

STATUS: BLOCKED




```
26-05-23 21:23:49,624 - INFO - 127.0.0.1 - - [23/May/2026 21:23:49] "ESC[32mPOST /register HTTP/1.1ESC[0
26-05-23 21:23:49,634 - INFO - 127.0.0.1 - - [23/May/2026 21:23:49] "GET /register HTTP/1.1" 200 -
26-05-23 21:39:59,246 - INFO - 127.0.0.1 - - [23/May/2026 21:39:59] "GET /login HTTP/1.1" 200 -
26-05-23 21:40:15,387 - WARNING - Failed login attempt 2/5 for: teststudent
26-05-23 21:40:15,415 - INFO - 127.0.0.1 - - [23/May/2026 21:40:15] "ESC[32mPOST /login HTTP/1.1ESC[0m"
26-05-23 21:40:15,421 - INFO - 127.0.0.1 - - [23/May/2026 21:40:15] "GET /login HTTP/1.1" 200 -
26-05-23 21:40:26,435 - WARNING - Failed login attempt 3/5 for: teststudent
26-05-23 21:40:26,459 - INFO - 127.0.0.1 - - [23/May/2026 21:40:26] "ESC[32mPOST /login HTTP/1.1ESC[0m"
26-05-23 21:40:26,465 - INFO - 127.0.0.1 - - [23/May/2026 21:40:26] "GET /login HTTP/1.1" 200 -
26-05-23 21:40:35,316 - WARNING - Failed login attempt 4/5 for: teststudent
26-05-23 21:40:35,340 - INFO - 127.0.0.1 - - [23/May/2026 21:40:35] "ESC[32mPOST /login HTTP/1.1ESC[0m"
26-05-23 21:40:35,345 - INFO - 127.0.0.1 - - [23/May/2026 21:40:35] "GET /login HTTP/1.1" 200 -
26-05-23 21:40:45,125 - WARNING - Failed login attempt 5/5 for: teststudent
26-05-23 21:40:45,155 - WARNING - Account LOCKED: teststudent
26-05-23 21:40:45,156 - INFO - 127.0.0.1 - - [23/May/2026 21:40:45] "ESC[32mPOST /login HTTP/1.1ESC[0m"
26-05-23 21:40:45,165 - INFO - 127.0.0.1 - - [23/May/2026 21:40:45] "GET /login HTTP/1.1" 200 -
```

6.4 CSRF Test

Attack Method:

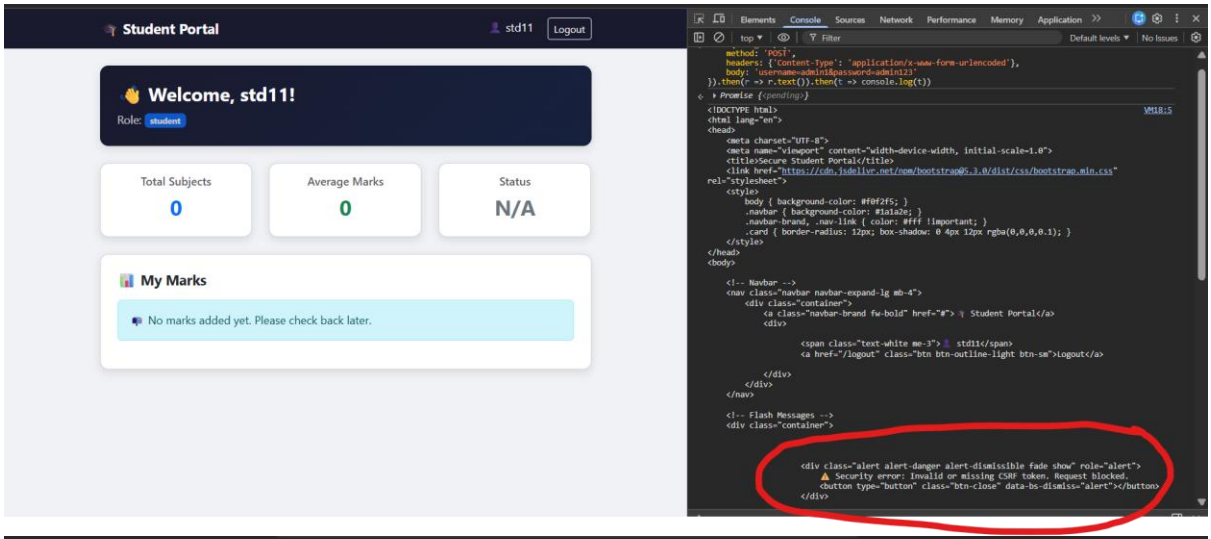
Used browser developer console to send a POST request without a CSRF token:

```
fetch('/login', {
    method: 'POST',
    headers: {'Content-Type': 'application/x-www-form-urlencoded'},
    body: 'username=admin1&password=admin123'
}).then(r => r.text()).then(t => console.log(t))
```

Actual Result:

Flask-WTF detected the missing CSRF token and returned: 'Security error: Invalid or missing CSRF token. Request blocked.'

STATUS: BLOCKED

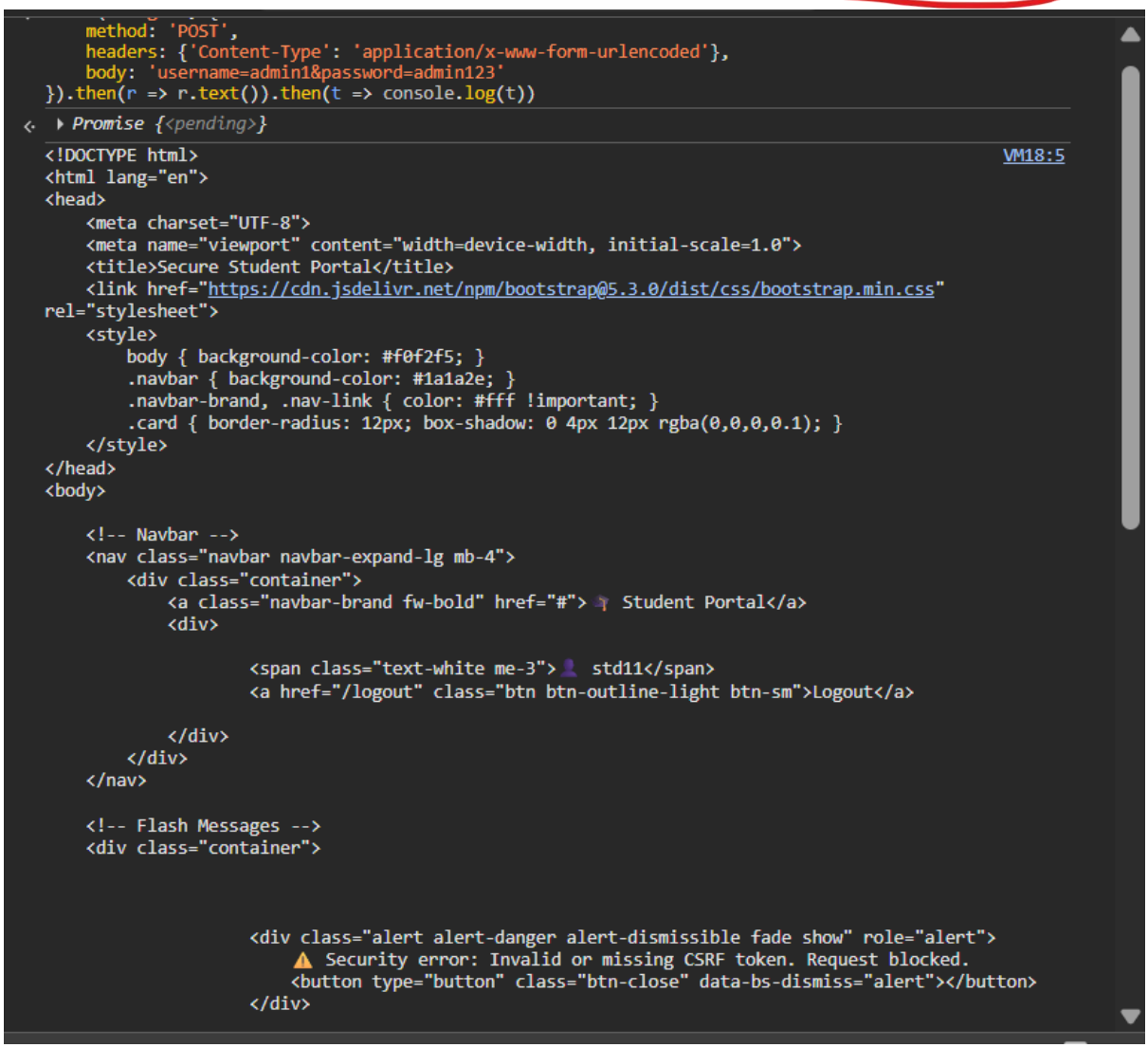

Student Portal
std11
Logout
Welcome, std11!
Role: student
Total Subjects
0
Average Marks
0
Status
N/A
My Marks
No marks added yet. Please check back later.
Elements
Console
Sources
Network
Performance
Memory
Application
method: 'POST',
headers: {'Content-Type': 'application/x-www-form-urlencoded'},
body: 'username=admin1&password=admin123'
}).then(r => r.text()).then(t => console.log(t))
Promise {<pending>}
VM18:5
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Secure Student Portal</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
<style>
body { background-color: #f0f2f5; }
.navbar { background-color: #1a1a2e; }
.navbar-brand, .nav-link { color: #fff !important; }
.card { border-radius: 12px; box-shadow: 0 4px 12px rgba(0,0,0,0.1); }
</style>
</head>
<body>
<!-- Navbar -->
<nav class="navbar navbar-expand-lg mb-4">

Student Portal

std11
Logout

</nav>
<!-- Flash Messages -->

Security error: Invalid or missing CSRF token. Request blocked.
<button type="button" class="btn-close" data-bs-dismiss="alert"></button>


method: 'POST',
headers: {'Content-Type': 'application/x-www-form-urlencoded'},
body: 'username=admin1&password=admin123'
}).then(r => r.text()).then(t => console.log(t))
Promise {<pending>}
<!DOCTYPE html>
VM18:5
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Secure Student Portal</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
<style>
body { background-color: #f0f2f5; }
.navbar { background-color: #1a1a2e; }
.navbar-brand, .nav-link { color: #fff !important; }
.card { border-radius: 12px; box-shadow: 0 4px 12px rgba(0,0,0,0.1); }
</style>
</head>
<body>
<!-- Navbar -->
<nav class="navbar navbar-expand-lg mb-4">

Student Portal

std11
Logout

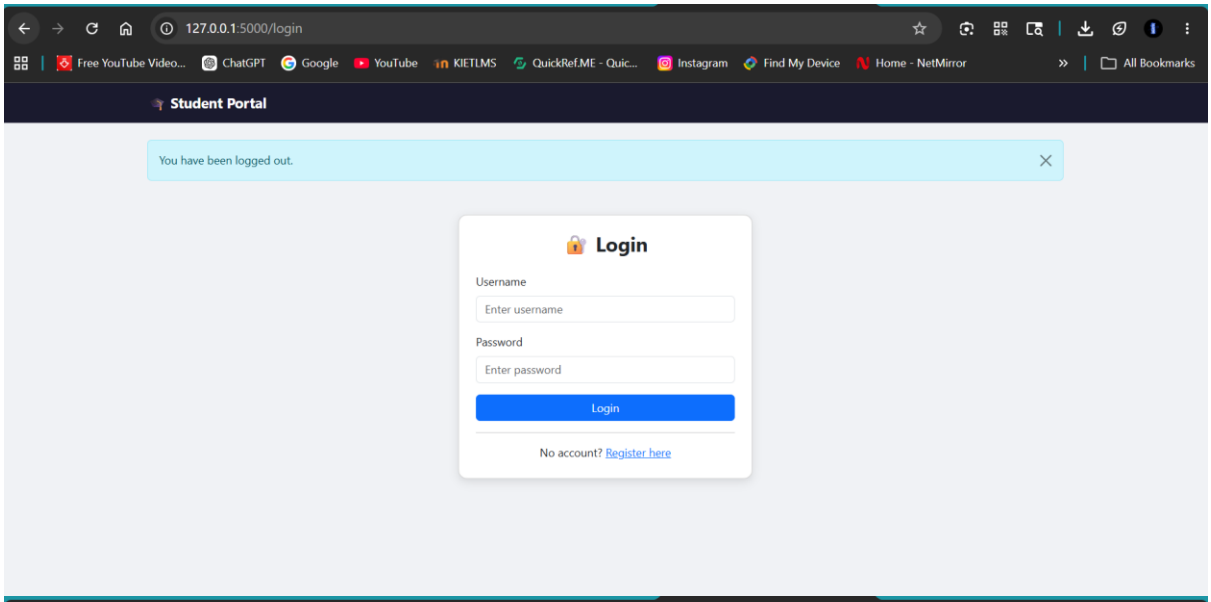
</nav>
<!-- Flash Messages -->

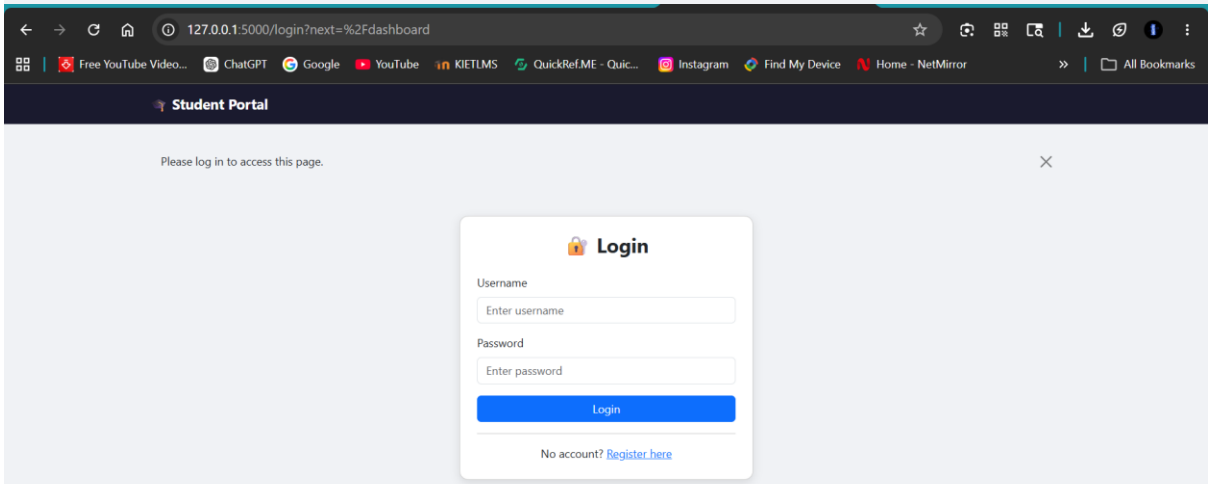
Security error: Invalid or missing CSRF token. Request blocked.
<button type="button" class="btn-close" data-bs-dismiss="alert"></button>

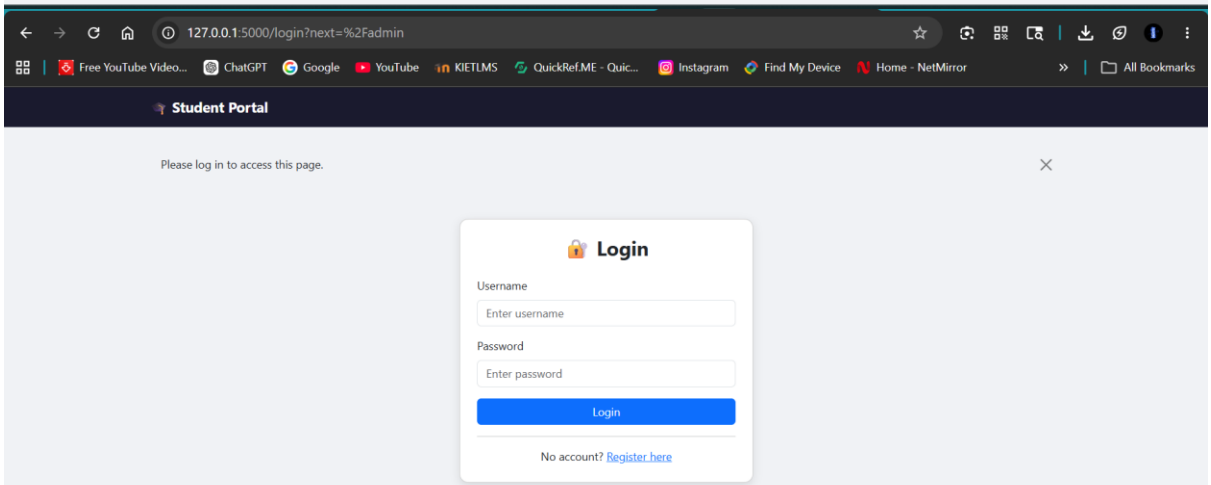
6.5 Unauthorized Access Test

Test	URL Accessed	Result
Access dashboard without login	/dashboard	Redirected to login page
Access admin panel without login	/admin	Redirected to login page
Student accessing admin panel	/admin (as student)	Access Denied - redirected
Admin accessing student dashboard	/dashboard (as admin)	Redirected to admin panel

STATUS: ALL BLOCKED


127.0.0.1:5000/login
Free YouTube Video...
ChatGPT
Google
YouTube
KIETLMS
QuickRef.ME - Quic...
Instagram
Find My Device
Home - NetMirror
All Bookmarks
Student Portal
You have been logged out.
Login
Username
Enter username
Password
Enter password
Login
No account? Register here


127.0.0.1:5000/login?next=%2Fdashboard
Free YouTube Video...
ChatGPT
Google
YouTube
KIETLMS
QuickRef.ME - Quic...
Instagram
Find My Device
Home - NetMirror
All Bookmarks
Student Portal
Please log in to access this page.
Login
Username
Enter username
Password
Enter password
Login
No account? Register here


127.0.0.1:5000/login?next=%2Fadmin
Free YouTube Video...
ChatGPT
Google
YouTube
KIETLMS
QuickRef.ME - Quic...
Instagram
Find My Device
Home - NetMirror
All Bookmarks
Student Portal
Please log in to access this page.
Login
Username
Enter username
Password
Enter password
Login
No account? Register here

7. Secure Design Principles Applied

Principle	How Applied in This Project
Least Privilege	Students only view their own marks. Admins cannot access student dashboard. Each role has minimum required permissions only.
Defense in Depth	Multiple security layers: input validation + prepared queries + RBAC + CSRF + brute force protection + logging. If one fails, others protect.
Fail Securely	On any error or invalid access, system redirects to login page with generic error messages. No sensitive information revealed.
Input Validation	All inputs validated server-side. Empty fields, invalid marks values, and short passwords are rejected before processing.
Separation of Concerns	Admin and student functionality completely separated. Every route individually protected with role checks.
Audit Logging	All security-relevant events logged with timestamps for accountability and incident response.

8. Conclusion

The Secure Student Portal successfully demonstrates the implementation of multiple security features in a real-world web application. The project covers all major aspects of Secure Software Development including threat modeling, secure coding practices, and penetration testing.

The application effectively prevents the following attacks:

- SQL Injection through SQLAlchemy ORM and prepared statements
- XSS through Jinja2 automatic output escaping
- Brute Force attacks through account lockout after 5 failed attempts
- CSRF attacks through Flask-WTF token validation
- Unauthorized access through Flask-Login and RBAC

The STRIDE threat modeling framework was used to systematically identify and address all major security threats. Security logging ensures accountability and provides an audit trail for all user actions.

This project demonstrates that security does not have to be complex. By using the right frameworks and following secure coding principles from the start, it is possible to build a secure and functional application.

9. References

1. Flask Documentation - https://flask.palletsprojects.com
2. Flask-Login Documentation - https://flask-login.readthedocs.io
3. Flask-WTF CSRF Protection - https://flask-wtf.readthedocs.io
4. bcrypt Documentation - https://pypi.org/project/bcrypt
5. OWASP Top 10 Web Application Security Risks - https://owasp.org/www-project-top-ten
6. STRIDE Threat Modeling - Microsoft Security Documentation
7. SQLAlchemy ORM Documentation - https://docs.sqlalchemy.org